

TaskOrbit Build & Deploy Documentation

Prerequisites

The following tools are required depending on which path you take.

Docker path (recommended):

- Docker Desktop — handles Python, Node.js, and PostgreSQL automatically. No other runtimes required.
- Git

Local development path (outside Docker):

- Python 3.11 (must be exactly 3.11; the backend is capped below 3.15 but 3.11 is the tested baseline)
- Poetry — Python dependency and virtualenv manager
- Node.js 20 or later with npm
- Git

Local Development

Running the Full Stack with Docker

This is the recommended path. Docker builds and starts all four services — no local Python or Node.js installation is needed.

Step 1. Clone the repository.

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

Step 2. Copy environment files.

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

Fill in `backend/.env` with API keys before starting the stack. See the Environment Variables section for the full list.

Step 3. Start the stack.

```
docker compose up --build
```

This starts four services:

- PostgreSQL on port 5432
- FastAPI backend on port 8000
- React frontend on port 5173
- LiveKit voice worker (connects outbound to LiveKit Cloud)

The first run takes several minutes while Docker pulls base images and builds layers. Subsequent starts use cached layers and are fast.

Step 4. Verify the backend.

```
curl http://localhost:8000/health
```

Expected response:

```
{"status": "ok", "service": "taskorbit-backend", "version": "0.1.0"}
```

Step 5. Open `http://localhost:5173` to access the frontend.

To stop all services:

```
docker compose down
```

To also remove the database volume (destroys all data):

```
docker compose down -v
```

Running the Backend Locally (Outside Docker)

Use this path when actively developing backend code and you want fast reload without rebuilding Docker images.

Start the database first:

```
docker compose up -d postgres
```

Then, in a separate terminal:

```
cd backend
poetry install
poetry run uvicorn taskorbit.api.main:app --reload --port 8000
```

The backend reloads automatically on file changes. The database runs in Docker while the application process runs on the host.

Running the Frontend Locally (Outside Docker)

Use this path when actively developing frontend code.

```
cd frontend
npm install
npm run dev
```

The frontend is available at `http://localhost:5173`. In development mode, all requests to `/api/*` are proxied to `http://localhost:8000`, so the backend must be running (either locally or via Docker).

Environment Variables

All credentials go in `backend/.env`. This file is gitignored — never commit it. Copy from `backend/.env.example` and fill in real values before starting the stack.

Variable	Purpose
DATABASE_URL	PostgreSQL connection string. Default: <code>postgresql://taskorbit:taskorbit@localhost:5432/taskorbit</code>
OPENAI_API_KEY	OpenAI API key. Required when using the OpenAI provider.
OPENAI_MODEL	OpenAI model name. Default: <code>gpt-4o-mini</code>

Variable	Purpose
GOOGLE_API_KEY	Google Gemini API key. Required when using the Gemini provider.
GOOGLE_MODEL	Gemini model name. Default: <code>gemini-2.5-flash</code>
DEEPGRAM_API_KEY	Deepgram speech-to-text key.
DEEPGRAM_MODEL	Deepgram model. Default: <code>nova-3</code>
DEEPGRAM_LANGUAGE	Language setting. Default: <code>multi</code> (auto-detect)
ELEVENLABS_API_KEY	ElevenLabs text-to-speech key.
ELEVENLABS_VOICE_ID	Voice ID. Default: <code>21m00Tcm4T1vDq8ikWAM</code> (Rachel)
ELEVENLABS_MODEL	TTS model. Default: <code>eleven_multilingual_v2</code>
LIVEKIT_URL	LiveKit server WebSocket URL (<code>wss://...</code>)
LIVEKIT_API_KEY	LiveKit API key.
LIVEKIT_API_SECRET	LiveKit API secret.

For the frontend, set `VITE_LIVEKIT_URL` in `frontend/.env.local` to the same LiveKit server URL.

Settings are loaded centrally via `taskorbit.config.get_settings()`. Do not read `os.environ` directly anywhere in the codebase.

Database

TaskOrbit uses PostgreSQL 16. The schema is managed via SQLAlchemy (async) and Alembic.

Start the database:

```
docker compose up -d postgres
```

Run all pending migrations:

```
cd backend
poetry run alembic upgrade head
```

The database schema includes the following tables: `conversations`, `conversation_messages`, `tool_executions`, `users`, `agent_configurations`.

To generate a new migration after changing SQLAlchemy models:

```
cd backend
poetry run alembic revision --autogenerate -m "describe the change"
```

Review the generated file in `backend/migrations/versions/` before applying it. Autogenerated migrations are not always complete — verify column types, constraints, and index definitions manually.

Testing

Backend Tests

Run the full test suite:

```
cd backend
poetry run pytest tests/ -v
```

All tests use mocks. No running database or external API keys are required.

Run linting and formatting checks:

```
poetry run ruff check .
poetry run ruff format --check .
```

Auto-fix issues:

```
poetry run ruff check --fix .
poetry run ruff format .
```

Frontend Tests

Run linting and formatting checks:

```
cd frontend
npm run lint
npm run format:check
```

Auto-fix issues:

```
npm run lint -- --fix
npm run format
```

Pre-commit Hooks

The repository ships pre-commit hooks that run the same checks automatically on every `git commit`.

Install the hooks:

```
poetry -C backend run pre-commit install
cd frontend && npm install
```

After installation, every commit automatically runs:

- Trailing whitespace and end-of-file checks
- `ruff check` and `ruff format` for Python files
- ESLint and Prettier for TypeScript files

If a hook auto-fixes a file, the commit aborts so you can review the change. Re-stage the fixed files and commit again:

```
git add -p
git commit -s -m "feat(scope): your message"
```

CI/CD Pipeline

Three GitHub Actions workflows run in parallel on every push and pull request.

Workflow	File	What it checks
Backend Linting	<code>backend-lint.yml</code>	Python install, <code>ruff check</code> , <code>ruff format --check</code> , <code>pytest</code>
Frontend Linting	<code>frontend-lint.yml</code>	Node install, ESLint, Prettier <code>--check</code>
Start Web Service	<code>start-web-service.yml</code>	Boots FastAPI, <code>curls /health</code> , runs <code>npm run build</code>

Jobs within each workflow chain with **needs:** so a failing stage short-circuits the rest of the run. The three workflows run in parallel with each other.

Branch protection on `main` requires all three pipelines to pass before a pull request can be merged.

Cloud Deployment

TaskOrbit deploys to Google Cloud Platform. Infrastructure is provisioned by Terraform; deployments are triggered by GitHub Actions.

Infrastructure

All infrastructure is defined in the `terraform/` directory. Provisioned resources:

- **Cloud Run** — separate services for the backend, frontend, and voice worker
- **Cloud SQL** — PostgreSQL 16 with private IP, regional high availability, and point-in-time recovery enabled
- **Artifact Registry** — stores Docker images for all three services
- **Secret Manager** — all API keys and credentials stored as versioned secrets, never embedded in container images
- **Cloud Scheduler** — stops Cloud SQL at 22:00 CET and restarts it at 07:00 Monday-Friday to reduce costs

Deployment Pipeline

Pushing to the `prod` branch triggers the full deployment pipeline:

1. GitHub Actions authenticates to GCP using Workload Identity Federation — no stored service account keys.
2. Docker images are built with `--target prod` and pushed to Artifact Registry.
3. Cloud Run services are updated to the new image revisions.

Do not push directly to `prod`. The correct flow is: develop on a feature branch, merge to `main`, verify on `main`, then open a merge request from `main` to `prod`.

Secrets

All sensitive values live in GCP Secret Manager under the prefix `taskorbit-`. They are injected as environment variables at Cloud Run instance startup.

To update a secret without triggering a full redeployment:

```
echo -n "new-value" | gcloud secrets versions add taskorbit-openai-api-key --data-file=
```

The new version takes effect on the next instance cold start. To force an immediate restart:

```
gcloud run services update taskorbit-backend --region europe-west1 --no-traffic
gcloud run services update-traffic taskorbit-backend --region europe-west1 --to-latest
```

Manual Deployment (Break-glass)

If the CI/CD pipeline is unavailable, deploy manually:

```
gcloud auth login
gcloud auth configure-docker europe-west1-docker.pkg.dev

docker build --target prod \
  -t europe-west1-docker.pkg.dev/PROJECT/taskorbit/backend:manual \
  ./backend

docker push europe-west1-docker.pkg.dev/PROJECT/taskorbit/backend:manual

gcloud run deploy taskorbit-backend \
  --image europe-west1-docker.pkg.dev/PROJECT/taskorbit/backend:manual \
  --region europe-west1
```

Replace PROJECT with the actual GCP project ID. Run equivalent commands for the frontend and voice worker services.

LiveKit Cloud Setup

The voice worker requires LiveKit Cloud credentials. These are separate from any LLM or STT/TTS keys.

1. Sign in at cloud.livekit.io and create or select a project.
2. Go to **Settings** → **API Keys** → **Create key**. Copy the key and secret immediately — the secret is shown only once.
3. Find the WebSocket URL under **Settings** → **Project** (format: `wss://your-project.livekit.cloud`).
4. Add to `backend/.env`:

```
LIVEKIT_URL=wss://your-project.livekit.cloud
LIVEKIT_API_KEY=your-key
LIVEKIT_API_SECRET=your-secret
```

5. Add to `frontend/.env.local`:

```
VITE_LIVEKIT_URL=wss://your-project.livekit.cloud
```

The backend mints short-lived JWTs (6-hour TTL) scoped to a single room and identity. The frontend uses these tokens to connect to LiveKit — it never receives or stores the API secret.

Observability

When running locally via Docker Compose, the observability stack starts automatically alongside the application services.

Service	URL	Notes
Prometheus	<code>http://localhost:9090</code>	Scrapes backend (:8000/metrics) and worker (:8001/metrics) every 5 seconds
Grafana	<code>http://localhost:3000</code>	Dashboards auto-provisioned on first boot; anonymous admin access in dev
Loki	internal only	Receives logs from both processes via Promtail

The Grafana dashboard (uid: `taskorbit-overview`) includes panels for:

- HTTP request rate and latency (p50, p95, p99)
- LLM request rate by provider
- Pipeline latency end-to-end
- Token usage over time
- Conversation error rates

In production, Prometheus and Grafana are not exposed publicly. Metrics are scraped internally within the VPC and dashboards are accessed via Cloud IAP or a VPN.